Blatt 05: Torch Riddle & Katzen-Café (Streams, JUnit, Optional<>, Visitor)

Carsten Gips (HSBI)

Unterlagen

- PDF-Version

---

1 Zusammenfassung

Auf diesem Blatt üben Sie den Umgang mit der Java-Stream-API und `Optional<>`. Sie erstellen erste JUnit-Tests und implementieren das Visitor-Pattern für ein einfaches Beispiel.

Hinweis: Bitte denken Sie daran, dass Sie spätestens seit Blatt 04 die Bearbeitung der Aufgaben in Ihren öffentlich sichtbaren Git-Repos durchführen sollen.
Erstellen Sie für **alle** Ihre Lösungen passende Pull-Requests gegen **Ihre eigenen Repos** (auch wenn dies nicht explizit in den Aufgaben gefordert wird).
Die Links zu Ihren Pull-Requests mit den Lösungen geben Sie bitte immer in Ihrem *Post Mortem* mit an.

2 Aufgaben

2.1 DevDungeon: Fackeln im Sturm (Umgang mit `Optional<>` und Streams)

Klonen Sie das Projekt DevDungeon und laden Sie es in Ihrer IDE als Gradle-Projekt. Betrachten Sie das Sub-Projekt "devDungeon". Dies ist ein von einem Studierenden (@Flamtky) erstelltes Spiel mit mehreren Leveln, in denen Sie spielerisch verschiedene Aufgaben *in-game* und *ex-game* lösen müssen.

Starten Sie den DevDungeon mit `./gradlew devDungeon:runDevDungeon`. Spielen Sie sich für diese Aufgabe durch das **zweite Level** ("Torch Riddle")[1].

Sie befinden sich in einem Raum mit Fackeln, welche Sie per Interaktion an- und ausschalten können. Neben jeder Fackel ist ein Briefkasten, der der Fackel einen Zahlenwert zuordnet. Irgendwo führt eine Tür zu einem zunächst versteckten Raum mit einer Belohnung - aber diese Tür geht erst auf, wenn Sie (a) die richtigen Fackeln an- bzw. ausgeschaltet haben, und wenn Sie (b) die defekte Methode `TorchRiddleRiddleHandler#getSumOfLitTorches` (im Package `level.devlevel.riddleHandler`) korrekt implementiert haben. Beachten Sie die entsprechenden Hinweise im Javadoc der Methode.

Das Tor zum nächsten Level geht unabhängig davon erst auf, wenn Sie den Boss-Gegner[2] in diesem Level besiegt haben … Hierzu ist *keine* Programmierung notwendig, lediglich geschicktes Spielen und gegebenenfalls rechtzeitiges Trinken von (dann hoffentlich vorhandenen) Heil-Tränken.

Hinweis: Aktuell ist das Projekt DevDungeon an einigen Stellen noch *Work-in-Progress*, beispielsweise fehlt häufig noch die Javadoc. Alle Gradle-Tasks, die von Checkstyle-Tasks abhängen (`checkstyleMain`, `check`,

[1] Das zweite richtige Level, also das zweite Level *nach* dem Demo-Level. Oder eben das dritte Level, wenn man das Demo-Level mitzählt :-)
[2] … sieht aus wie eine wandelnde Kerze …

`build`, ...) werden deshalb fehlschlagen. Sie können den DevDungeon aber wie oben beschrieben mit `./gradlew devDungeon:runDevDungeon` (bzw. über den Task `devDungeon:runDevDungeon` aus der IDE heraus) starten.

WICHTIG: Achten Sie bitte darauf, dass im Projektpfad **keine Leerzeichen** und keine Sonderzeichen (Umlaute o.ä.) vorkommen! Dies kann zu seltsamen Fehler führen. Bitte auch darauf achten, dass Sie als JDK ein **Java SE 21 (LTS)** verwenden. Unter Windows ist der Einsatz von WSL empfehlenswert.

2.2 Katzen-Café

Forken Sie das "Cat-Cafe"-Repo und erzeugen Sie sich eine lokale Arbeitskopie von Ihrem Fork.

2.2.1 Code-Analyse

Analysieren Sie die Modellierung des Binärbaums (`Tree`, `Empty`, `Node`) und erklären Sie die Funktionsweise:

- Was sind Vorteile, was sind Nachteile dieser Modellierung?
- Was musste getan werden, um die selbst implementierten Bäume in Schleifen (`Tree<X> mytree; for (Tree<X> t: mytree){...}`) und in Streams `Tree<X> mytree; mytree.stream().... nutzen zu können?
- Wie funktioniert der `TreeIterator`?

2.2.2 Umgang mit `Optional<>`

Bauen Sie die beiden Methoden `CatCafe#getCatByName` und `CatCafe#getCatByWeight` so um, dass ein passendes `Optional<>` zurückgeliefert wird. Passen Sie die entsprechenden Methodenaufrufe in `Main#main` entsprechend an.

Tipp: Stellen Sie in den beiden Methoden auf die Java-Stream-API um, dann ergibt sich die Nutzung von `Optional<>` fast von selbst.

2.2.3 JUnit

Erstellen Sie mit JUnit 4 oder 5 mindestens 10 unterschiedliche Testfälle für die Klasse `CatCafe`.

Tipp: In der Gradle-Konfiguration der Vorgabe ist bereits JUnit5 konfiguriert, d.h. die entsprechenden Abhängigkeiten werden durch Gradle aufgelöst. Wenn Sie die Vorgaben als Gradle-Projekt in Ihrer IDE öffnen, dann steht Ihnen dort auch die JUnit5-Bibliothek automatisch zur Verfügung. Wenn Sie JUnit4 nutzen möchten, müssten Sie bitte die Gradle-Konfiguration entsprechend anpassen. Mit `./gradlew test` können Sie entsprechende Testfälle ausführen.

2.2.4 Visitor-Pattern

Die Klasse `CatCafe` hat eine Methode `CatCafe#accept`, die einen Visitor mit dem parametrischen Typ `TreeVisitor` an das intern genutzte Feld `Tree<FelineOverLord> clowder` weiterleitet.

Implementieren Sie das Visitor-Pattern für den Baum (`Tree`), indem Sie das Interface `TreeVisitor` implementieren:

1. Erstellen Sie einen konkreten Visitor `InOrderVisitor`, der den Baum **inorder** traversiert.
2. Erstellen Sie einen weiteren konkreten Visitor `PostOrderVisitor`, der den Baum **postorder** traversiert.

Beim Besuch eines Knotens soll jeweils die Methode `toString()` für den Datenanteil aufgerufen werden und passend mit den Ergebnissen der Traversierung der linken und rechten Teilbäume konkateniert werden und der resultierende String zurückgeben werden.

Fügen Sie passende Aufrufe der beiden Visitoren in `Main#main` hinzu.

3 Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung oder bis zu 3er Teams
- Abgabe:
  - Post Mortem im ILIAS eintragen:

    Verfassen Sie im ILIAS pro Blatt und pro Team-Mitglied ein aussagekräftiges und nachvollziehbares "*Post Mortem*". Gehen Sie dabei auf folgende Punkte ein:

    1. Zusammenfassung: Was wurde gemacht?
    2. Implementierungsdetails: Kurze Beschreibung besonders interessanter Aspekte der Umsetzung.
    3. Was war der schwierigste Teil bei der Bearbeitung? Wie haben Sie dieses Problem gelöst?
    4. Was haben Sie gelernt oder (besser) verstanden?
    5. Team: Mit wem haben Sie zusammengearbeitet?
    6. Links zu Ihren Pull-Requests mit der Lösung.

    Das Post Mortem muss von **jeder Person** im Team **individuell** verfasst und abgegeben werden. Der Umfang des Textes soll zwischen 200 und 400 Wörtern liegen.

    Laden Sie hier bitte **nicht** Ihre Lösungen hoch!
  - Deadline: 30. Mai, 08:00 Uhr
- Vorstellung im Praktikum: 30. Mai

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 4abb344 (homework: point windows users to 'windows subsystem for linux' (wsl) for dungeon tasks, 2025-05-27)